Final Answer Correctness

Introduction

The goals of this evaluation:

1. Compare Message final answer vs an expected answer.

Results on a sample of Frames

based on Wiki Sample Based on FRAMES, the following is a summary of the first 12 queries for each of the planners: Strategic, Deep, and Priority:

Unified Planners - final answer correctness.xlsx

Setup

Use FRAMES, HE FRAMES and ReFRAMES agents and datasets (FRAMES (EN/HEB) Agent and Data).

For each agent above, ask all the dataset questions in a new channel (also use separate channels for strategic planner and for priority planner).

Test Flow

1. Query the EN FRAMES agent, using **Strategic Planner and after that with the Priority Planner**
2. Given a DB name, iterate over all Messages collection, using some mongo filer (default: all messages).
3. For every message that has a key "evaluation.expected_answer" or if a collection '`'correctness_frames'`' exists (a collection with all the ground truth):
   a. Compare the actual answer with the expected answer, using the metrics below
   b. Store the results in "evaluation.correctness"

Metric

The following two groups of metrics should be collected:

RAGAS

The **Answer Accuracy, Factual Correctness** and **Faithfulness** (please see List of available metrics - Ragas for detailed definition).

- **Answer Accuracy** - examines how the final response is aligned with the "evaluation.expected_answer". Store the result under "evaluation.correctness.answer_accuracy"
- **Factual Correctness** - compares and evaluates the factual alignment of the final response with the "evaluation.expected_answer".
- **Faithfulness** - examines how factually consistent the final response with the `retrieved context`. "evaluation.correctness.faithfulness" (apply on the actual context that was used to compose the answer).

Additional useful metrics:

- `answer_relevancy` : Is the answer relevant to the question?
- `answer_correctness` : Does the answer match the ground truth?

Ragas usage example can be found here:

```python
from datasets import Dataset
from ragas import evaluate
from ragas.metrics import (
    faithfulness,
    answer_accuracy,
    factual_correctness
)

# Example data:
# "generated_response" - the final responces
# "retrieved_documents" - the content of the retreived documents
data = {
    "query": ["What is the capital of France?"],
    "generated_response": ["Paris is the capital of France."],
    "retrieved_documents": [["Paris is the capital of France. It is a major European city known for its culture."]]
}

# Convert the data to a Hugging Face Dataset
dataset = Dataset.from_dict(data)

# Define the metrics you want to evaluate
metrics = [
    faithfulness,
    answer_accurecy,
    factual_correctness
]

# Evaluate the dataset using the selected metrics
results = evaluate(dataset, metrics)

# Display the results
for metric_name, score in results.items():
    print(f"{metric_name}: {score:.3f}")
```

For more details about RAGAS usage please see: 📐 Evaluate a simple LLM application - Ragas

NLI Factual Evaluation

Use the definition from the regression evaluation (Human vs LLM Annotation | Setup)

- Collect the metrics: `entailment_nli_precision` and `entailment_nli_recall`.

Test Flow - EN

1. Iterate over all the queries in the FRAMES dataset:
   a. Query the EN FRAMES agent, using **Strategic Planner and after that with the Priority Planner**
   b. Read the returned final answer of a specific message
   c. Evaluate the returned final answer vs the expected answer, using the defined metrics above
   d. For the message sent, store in Mongo, in the messages collection the following values:

```python
"correctness_final_answer_evaluation": {
    "faithfulness": ...,
    "answer_accuracy": ...,
    "factual_correctness": ...,
    "entailment_nli_precision": ...,
    "entailment_nli_recall": ...,
},
"evaluation.correctness": {
  "expected_answer": ... # FRAMES expected answer
  "expected_nodes": ... # FRAMES expected relevany pages
}
```

Running

1. Upload the FRAMES golden-set

- In MongoDB, create the following DB→Collection pair:

  `correctness_frames → full_frames_with_entities`

- Upload the following
  frames_with_entities.csv (for full weekly evaluation) and FRAMES_sample.csv (for daily evaluation) to the

`full_frames_with_entities` collection prepared in MongoDB.

---

2. Prepare a test conversation

- Each evaluation run should target a specific conversation channel (DB), corresponding to a different kind of planner: Strategic/ Deep/ Priority. Each conversation should be associated with **Wiki Sample** data source agent.
- **Do not reuse the same channel for repeated tests** — create a new channel or collection for each new evaluation to avoid mixed results.

---

3. Run all queries

- Use either `frames_with_entities.csv` (weekly) or `FRAMES_sample.csv` (daily) to run all prompts one by one automatically.
- Under your selected conversation channel, ensure all these user queries (prompts) are recorded in the `messages` collection (they should be there since they are automatically created after a query is processed).
- Only queries where `step_id` is `null` will be evaluated since they correspond to the original query inserted by the user, not the proceeding steps.

---

4. Invoke the evaluation pipeline

There are 2 ways to run the evaluation pipeline:

1. Recommended: using `correctness.run_final_answer_correctness` that read a JSON file containing `'conversation_db_name'` and `'db_filter'`, and calls `final_answer_correctness.compute_final_answer_correctness_metrics`
2. Alternatively, you can run the pipeline directly using the Python entry point in `final_answer_correctness`:

   ```
   if __name__ == '__main__':
       test_compute_final_answer_correctness_metrics("<conversation_db_name>")
   ```

   Replace `<conversation_db_name>` with the target MongoDB conversation DB you want to evaluate.

   Example:

   ```
   python final_answer_correctness.py
   ```

---

5. What it does

- Loads the frames golden-set (`full_frames_with_entities`)
- Loads the user queries (`messages` where `step_id` is `null`)
- Compares each query's response with its ground truth using NV-style scoring
- Saves the scores into: `evaluation → final_answer_correctness`
- Generates an Excel file: `final_answer_correctness.xlsx` with all scores
- Generates a collection called `final_answer_correctness` within the conversation DB, and saves all the scores.

Note

- The scoring runs fully automated using the LLM backend configured in the environment.
- For any update to the scoring prompt or rules, update `PromptType.RAGAS_WO_NV_ACCURACY` and `NvAccuracyScoreWO`.

Final Answer Correctness Evaluation Pipeline

This section documents the **design and implementation** of a custom pipeline for evaluating the correctness of generated answers by comparing them to ground truth answers, scoring them using a custom NV-style scoring prompt, and storing the results for traceability and quality tracking.

📚 **Source Data**

Component	Location
Ground Truth DB	`correctness_frames`
Ground Truth Collection	`full_frames_with_entities`
User Queries DB	(passed at runtime)
User Queries Collection	`messages`
Filter	Only messages where `step_id` is `null` (i.e., primary user queries, not intermediate chain-of-thought steps).

Key fields:

- `query` — the user's question.
- `response` — the system-generated answer.
- `Answer` — the expected correct answer (from `full_frames_with_entities`).

🔁 **Processing Workflow**

1️⃣ Load Ground Truth

Load the reference answers from `correctness_frames.full_frames_with_entities`.

2️⃣ Load User Queries

Load all user queries from the target conversation DB's `messages` collection, filtered where `step_id` is `null`.

3️⃣ For Each Query:

- Match the query to its ground truth in the frames.
- If no match, skip with a log.
- If the response is `"I could not find an answer to this query"`, assign a correctness score of **0**.

- Otherwise:
  - Call the custom NV-style LLM prompt **twice**:
    - once comparing `response` vs. `ground truth`
    - once comparing `ground truth` vs. `response`
  - Average the two prompt scores and normalize (divide by 8.0) for final scoring.
- Log the computed score.
- Collect `query`, `response`, `ground_truth`, and the final numeric `score`.

4️⃣ **Store the Results:**

- Insert results into the `evaluation.final_answer_correctness` collection.
- Also export an Excel `.xlsx` file (`final_answer_correctness.xlsx`) including:
  - All records.
  - A final row showing the **average score**.

📐 **Evaluation Method**

- **Scoring Logic:**
  - Uses a custom JSON output parser (`NvAccuracyScoreWO`) to enforce a numeric rating: `0`, `2`, or `4`.
  - Uses a dedicated `PromptType` (`RAGAS_WO_NV_ACCURACY`) to instruct the LLM to rate factual alignment.
- **Final Score:**
  - Calculated as: `(score1 + score2) / 8.0`
    (since each score is one of `{0, 2, 4}` → the sum is in `{0, 2, 4, 6, 8}`).

📑 **Output**

Component	Location
Database	`evaluation`
Collection	`final_answer_correctness`
Excel File	`final_answer_correctness.xlsx`

Example stored document:

```
1 json
```

CopyEdit

```
{   "query": "...",   "response": "...",   "ground_truth": "...",   "score": 0.875 }
```

The Excel export includes an extra row at the end showing the **mean score** for all evaluated records.

💡 Key Constants

These are declared at the top of the code for clarity and reuse:

- `GOLDENSET_DB = 'correctness_frames'`
- `FULL_FRAMES_COLLECTION = 'full_frames_with_entities'`
- `EVAL_DB = 'evaluation'`
- `EVAL_COLLECTION = 'final_answer_correctness'`

📈 Possible Future Improvements

✅ Replace the ground truth-only prompt with real retrieved context passages.
✅ Add versioning or timestamp fields to track evaluation runs.
✅ Build dashboards to monitor correctness scores over time.
✅ Add deduplication logic to avoid duplicate DB writes.

📐 Appendix: Additional Evaluation Metrics

The following RAGAs metrics are computed for each response:

Metric	Description
`answer_relevancy` -X	Is the answer relevant to the question?
`faithfulness` -X	Is the answer factually consistent with the given context?
`answer_correctness` -V	Does the answer match the ground truth? `answer_accuracy` is better.
`answer_accuracy` -V	An LLM-based assessment of the factual accuracy of the answer.
`factual_correctness` -V	Does the answer include factual errors even if it seems plausible? This metric identifies hallucinations and unsupported claims using an LLM-based evaluator.

🧾 Output Storage

Evaluated records are saved in a new MongoDB location:

- **Database:** `evaluation`
- **Collection:** `final_answer_correctness`

Each stored document includes:

```
{
  "query": "...",
  "response": "...",
  "ground_truth": "...",
```

```
  "scores": {
    "answer_relevancy": 0.91,
    "faithfulness": 0.85,
    ...
  }
}
```